DEDICATED TO INNOVATION

www.bacancy.com

BACANCY®

Customer Satisfaction Is Our Highest Priority

Employee Matters

In this course

What we are learning in this session

- Introduction
- Query Syntax and Method Syntax
- Filtering and Sorting
- Projections
- Aggregation Functions
- Joins
- Grouping and Nested Queries
- Deferred vs Immediate Execution

Let’s Start

Linq - Introduction

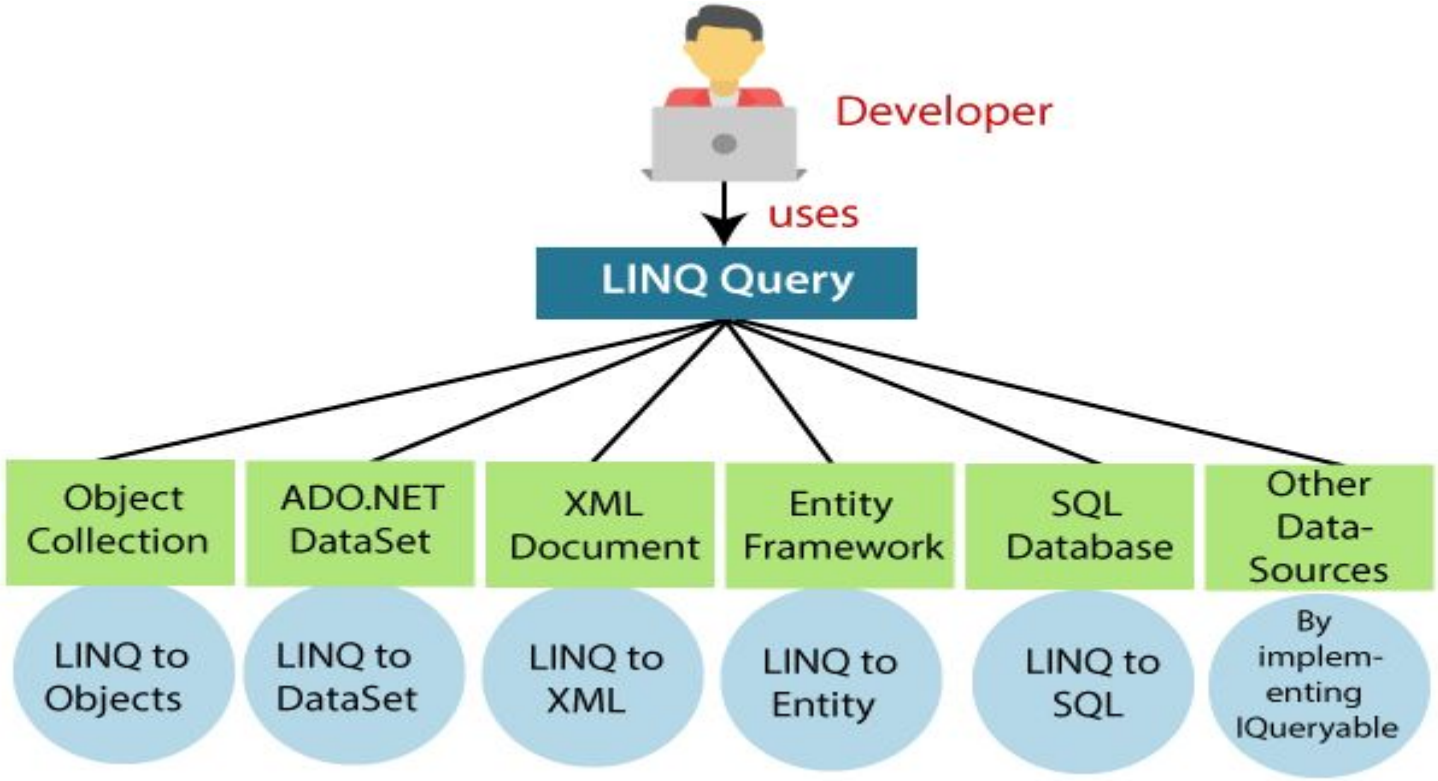
- **LINQ**, which stands for **Language Integrated Query**, is a feature in programming languages like C# and VB.NET that allows developers to write queries directly within the programming language to interact with data.

- LINQ provides a consistent model for querying different types of data sources, such as databases, collections, XML, and more.

- Example : Query a list of numbers in C#

```
int[] numbers = { 1, 2, 3, 4, 5 };
var evenNumbers = numbers.Where(n => n % 2 == 0);
```

Linq - Usage


Developer
uses
LINQ Query
Object Collection
ADO.NET DataSet
XML Document
Entity Framework
SQL Database
Other Data-Sources
LINQ to Objects
LINQ to DataSet
LINQ to XML
LINQ to Entity
LINQ to SQL
By implem-enting IQueryable

Linq - Query Syntax

- Query syntax is **similar to SQL.**
- Starts with ***from*** clause and can be end with ***Select*** or ***Group By*** clause.

```
//Data Source
List<int> integerList = new List<int>()
{
    1, 2, 3, 4, 5, 6, 7, 8, 9, 10
};
//LINQ Query using Query Syntax
var QuerySyntax = from obj in integerList
                  where obj > 5
                  select obj;

//Execution
foreach (var item in QuerySyntax)
{
    Console.Write(item + " ");
}
```

Initialization

Condition

Selection

Linq - Method Syntax

- Method syntax like ***calling*** an ***extension*** method already included.
- It uses a lambda expression to define the condition for the Query and implicitly called using variable - var to hold result of the query.

```
// String Collection
IList<string> stringList = new List<string>() {
    "May",
    "June",
    "July",
    "August"
    };

// Method Query
var result = stringList.Where(s => s.Contains("May"));
```

Lambda Expression

Extension Method

Linq - Filtering and Sorting

- **Filtering:** *Where()* method filters elements.

```
var filteredNumbers = numbers.Where(n => n > 2);
```

- **Sorting:** data with *OrderBy*, *OrderByDescending*, *ThenBy*, *ThenByDescending*

```
var sortedNumbers = numbers.OrderBy(n => n);
var descSortedNumbers = numbers.OrderByDescending(n => n);
```

- Working with **nullable values** in queries

```
List<int?> nullableNumbers = new List<int?>
                        {1, null, 3, 5, null};
var nonNullNumbers = nullableNumbers.Where(n => n.HasValue)
                    .Select(n => n.Value);
```

Projection in Linq

- **Select:** Selecting specific properties using *Select*():

```
IEnumerable<string> list = stringList.Select(s => s);
IEnumerable<int> lengthList = stringList.Select(s => s.Length);
```

- **Select Many:** Flattening Collection using *SelectMany()*
  - Applied on nested collections.

```
IEnumerable<char> characterList = stringList.SelectMany(s => s);
```

- Creating **Anonymous** Types:

```
var lengthProjection = stringList.
                       Select(s => new { Month = s, Length = s.Length });
```

Linq - Aggregations Functions

- Using aggregation methods like *Count*, *Sum*, *Average*, *Min*, *Max*

```
int count = numbers.Count();
int sum = numbers.Sum();
```

- Grouping data with *GroupBy*

```
var groupedEmployees = employees.GroupBy(e => e.Department);
```

- Selecting aggregated results in projections

```
var departmentStats = employees.GroupBy(e => e.Department)
                               .Select(g => new
                               {
                                   Department = g.Key,
                                   EmployeeCount = g.Count(),
                                   AverageSalary = g.Average(e => e.Salary)
                               });
```

Linq - Grouping Queries

- Grouping data using *GroupBy* and *ToLookup*

```
var groupedEmployees = employees.GroupBy(e => e.Department);
var lookupEmployees = employees.ToLookup(e => e.Department);
```

- Aggregating grouped data with custom selectors

```
var departmentStats = employees.GroupBy(e => e.Department)
    .Select(g => new
    {
        Department = g.Key,
        EmployeeCount = g.Count(),
        MaxSalary = g.Max(e => e.Salary)
    });
```

- Creating Nested Queries and Subqueries in LINQ

```
var departmentsWithManyEmployees = employees.GroupBy(e => e.Department)
            .Where(g => g.Count() > 3)
            .Select(g => new
            {
                Department = g.Key,
                Employees = g.Select(e => e.Name) // Subquery to get employee names
            });
```

Linq - Set Operations

- **Distinct:** Removes duplicates from collections:

```
List<int> integerListFirst = new List<int>{ 1, 2, 3, 4, 5, 4 };
List<int> integerListSecond = new List<int> { 4, 5, 6, 7, 8 };
var distinct = integerListFirst.Distinct();
```

- **Union:** Combines two sequences, removing duplicates:

```
var union = integerListFirst.Union(integerListSecond);
```

- **Intersect:** Finds Common elements between two sequences:

```
var intersect = integerListFirst.Intersect(integerListSecond);
```

- **Except:** Returns elements from the first sequence not found in the second:

```
var except = integerListFirst.Except(integerListSecond);
```

Linq - Deferred vs Immediate Execution

- **Deferred Execution:** LINQ queries are not executed when they are defined. Instead, they are executed when the result is enumerated.

```
var numbers = new List<int> { 1, 2, 3, 4, 5 };

// Deferred execution
var query = numbers.Where(n => n > 2);
// Data changes
numbers.Add(6);

// Query is executed here
foreach (var num in query){
    Console.WriteLine(num);
}
```

- Queries are re-evaluated each time they are enumerated.
- Useful for working with large datasets or when the data might change before execution.

Linq - Deferred vs Immediate Execution

- **Immediate Execution:** Forces the query to execute immediately and return the result as a collection (e.g., ToList, ToArray, ToDictionary)

```
// Immediate execution using ToList
var result = numbers.Where(n => n > 2).ToList();

// Data changes
numbers.Add(6);

// Query is already executed, so changes are not reflected
foreach (var num in result)
{
    Console.WriteLine(num);
}
```

- Give results immediately and don't want the query to be re-evaluated.
- Useful for caching or small datasets.

Linq - Resources

Resource Link:

- https://learn.microsoft.com/en-us/dotnet/csharp/linq/
- https://dotnettutorials.net/lesson/introduction-to-linq/
- https://www.javatpoint.com/linq

Q&A

Thank you